参考答案与评分标准

操作系统模拟试卷 A · 共 100 分

一、单项选择题（30 分）

每题 2 分。错选、漏选或多选均不得分。

题号	答案	解析
1	C	操作系统承上启下，既管理硬件，又为应用提供抽象、接口和隔离。
2	A	库可管理已取得的内存；需要向内核申请更多内存时才可能触发相关系统调用。
3	D	微内核将部分服务移至用户态，增强模块化和故障隔离，但带来通信开销。
4	B	事件完成使进程具备运行条件；仍须经过调度才能进入运行态。
5	C	ELF 的执行视图使用段，LOAD 段描述待加载的代码和数据。
6	A	线程各自维护 PC、SP 和栈；栈仍位于共享地址空间，并不自动具备线程间内存隔离。
7	D	多对一映射只有一个内核调度实体；阻塞该实体也可能使其他用户级线程无法继续执行。
8	B	持续到达的短任务可能不断抢占或挤占长任务，造成长任务饥饿。

逐题课件出处见文末索引及可编辑答案 Markdown；解答题亦附主要出处。

一、单项选择题（续）

每题 2 分。错选、漏选或多选均不得分。

题号	答案	解析
9	A	小时间片有利于交互响应，但过多切换会消耗 CPU 时间。
10	C	提升持锁者的优先级可缓解优先级反转；相关锁释放后应按规则恢复优先级。
11	D	本地关中断只约束本核；跨核共享数据通常仍需原子指令和锁等机制。
12	B	TestAndSet 的读旧值与置位不可分割，首次成功者取得锁；它不自动保证公平性。
13	C	读者流可能持续保持读者数量非零，导致写者饥饿；互斥规则本身仍可成立。
14	A	安全要求存在可完成序列；不安全只表示无法保证后续安全，不能直接断定已死锁。
15	D	共享内存提供数据通路，互斥与条件同步仍需另行设计。

逐题课件出处见文末索引及可编辑答案 Markdown；解答题亦附主要出处。

二、填空题（20 分）

每题 2 分；规范同义表述可得分。符号题须表达完整关系。

题号	参考答案	说明
16	进程控制块（PCB）	写出“进程控制块”或“PCB”均可。
17	3	仅针对题目明确给定的记录型实现；负值的绝对值表示等待者数量。
18	Max − Allocation	按资源种类逐项相减。
19	0	初值为 0，使 B 在 A 发出通知前无法越过 P 操作。
20	Mesa	区别于唤醒后立即移交执行权的 Hoare 语义。
21	丢弃（或：不保存／忽略）	空队列上的 signal 等同空操作，条件变量本身不保存通知次数。
22	N	所有参与者到齐后再放行。
23	读侧临界区	等价地写“经过覆盖这些读者的宽限期”可得分；无需等待此后所有新读者都停止。
24	fd[0]	fd[1] 为写端。
25	accept（或 accept()）	listen 建立监听状态；accept 返回用于该连接的新套接字。

26. 系统调用、进程状态与上下文（10 分）

（1）系统调用入口（4 分）

用户代码或库封装按 ABI 放置系统调用号及参数，再执行陷入指令；处理器转至内核入口。入口建立／切换至该线程的内核栈，保存必要的用户执行上下文；内核校验系统调用号和参数，按系统调用表分发到 read 的实现。传入用户缓冲区等参数时，内核应校验访问范围。具体保存工作由硬件与软件按架构分工完成。

（2）阻塞、唤醒与调度（4 分）

事件	状态变化
P 等待尚未完成的磁盘 I/O	P：运行 → 阻塞
调度 Q	Q：就绪 → 运行
P 等待的 I/O 完成	P：阻塞 → 就绪
P 再次获得 CPU	P：就绪 → 运行

I/O 完成只使 P 重新具备运行条件。是否立即抢占 Q，取决于调度策略、优先级等；不能把唤醒直接等同于立即运行。

（3）两者不等价（2 分）

模式切换改变特权级，可以仍由 P 使用 CPU、完成内核服务后返回 P；进程上下文切换则保存 P 的执行状态并恢复 Q 的执行状态，改变当前调度实体。一次系统调用不一定引起进程切换，本题因 I/O 等待才需要阻塞和调度。

评分标准：（1）调用号／参数与陷入、内核入口／内核栈、保存上下文、校验与分发各 1 分；（2）P 阻塞 1 分，Q 被调度 1 分，P 被唤醒并随后被调度 1 分，解释不一定立刻运行 1 分；（3）不等价且区分两种机制各 1 分。

课件出处：Intro02 操作系统结构与组成.pdf，PDF 第 18、23、27—28、35 页；Proc01 进程.pdf，PDF 第 18—19、24—28 页；Proc03 CPU调度.pdf，PDF 第 6—7 页。

27. SRTCF 与 RR 调度计算（10 分）

（1）SRTCF（5 分）

```
时间区间   [0,1) [1,3) [3,4) [4,6) [6,10) [10,16)
运行进程      P1    P2    P3    P2     P4      P1
```

t=3 时，P2 剩余 2，P3 只需 1，故 P3 抢占；t=5 时，P2 只剩 1，P4 到达但不抢占。

进程	完成时刻	周转时间	等待时间
P1	16	16	9
P2	6	5	1
P3	4	1	0
P4	10	5	1

平均周转时间 = (16+5+1+5)/4 = 6.75 ms；平均等待时间 = (9+1+0+1)/4 = 2.75 ms。

（2）RR，q=2（5 分）

```
时间区间   [0,2) [2,4) [4,6) [6,7) [7,9)
运行进程      P1    P2    P1    P3    P2
时间区间   [9,11) [11,13) [13,15) [15,16)
运行进程       P4      P1      P4      P1
```

t=6 时，就绪队列依次为 P3、P2、P4、P1。P3 只执行 1 ms 即完成，调度器随即运行 P2。

进程	完成时刻	周转时间	等待时间
P1	16	16	9
P2	9	8	4
P3	7	4	3
P4	15	10	6

平均周转时间 = (16+8+4+10)/4 = 9.50 ms；平均等待时间 = (9+4+3+6)/4 = 5.50 ms。

评分标准：每种算法各 5 分。完整运行顺序与切换边界 2 分，完成时刻 1 分；平均周转时间、平均等待时间各 1 分。因前项单处算错而导致的后续误差，可对正确公式和一致计算酌情给过程分，不重复扣尽。

课件出处：Proc03 CPU调度.pdf，PDF 第 9、15—20 页。

28. 用信号量实现有界缓冲区（10 分）

（1）初始化与正确伪代码（6 分）

```
semaphore empty = N;
semaphore full  = 0;
semaphore mutex = 1;

producer(x) {
    P(empty);
    P(mutex);
    put(x);
    V(mutex);
    V(full);
}

consumer() {
    P(full);
    P(mutex);
    x = get();
    V(mutex);
    V(empty);
    return x;
}
```

empty 记录尚未被预订的空位许可，full 记录可消费的项目许可；mutex 保护缓冲区及队头、队尾等共享状态。等待资源许可应放在取得 mutex 之前，避免持有缓冲区互斥锁睡眠等待空位或数据。生产与消费项目本身的处理可以放在缓冲区临界区之外。

（2）错误顺序可导致死锁（4 分）

缓冲区已满，empty=0。生产者先取得 mutex，再执行 P(empty) 而阻塞，却仍持有 mutex。消费者虽然能取得一个 full 许可，却在 P(mutex) 处阻塞，无法执行 get()，也就无法 V(empty)。生产者等待消费者产生空位，消费者等待生产者释放 mutex，形成相互等待。

评分标准：（1）三个初值 2 分，正确生产者流程 2 分，正确消费者流程 2 分；同步含义正确的等价实现可得分。（2）指出生产者持 mutex 等空位 1 分，消费者等待 mutex 1 分，无法消费并释放空位 1 分，形成相互等待／死锁 1 分。

课件出处：Sync03 信号量-条件变量-屏障.pdf，PDF 第 5—10 页；Sync02 死锁.pdf，PDF 第 9 页。

29. 银行家算法与安全状态（10 分）

（1）Need = Max − Allocation（3 分）

进程	Need
P0	(0,2,0)
P1	(2,0,1)
P2	(1,1,1)
P3	(0,0,2)

（2）当前状态安全（3 分）

一个安全序列是 P2 → P0 → P1 → P3。安全性检查只模拟可完成性，不实际改变原状态。

模拟步骤	完成前满足 Need ⩽ Work	归还 Allocation 后的 Work
初始	—	(1,1,1)
P2 完成	(1,1,1) ⩽ (1,1,1)	(2,2,2)
P0 完成	(0,2,0) ⩽ (2,2,2)	(3,2,3)
P1 完成	(2,0,1) ⩽ (3,2,3)	(4,3,3)
P3 完成	(0,0,2) ⩽ (4,3,3)	(5,4,4)

（3）不能立即批准该请求（3 分）

Request0=(0,1,0) ⩽ Need0=(0,2,0)，且 Request0 ⩽ Available=(1,1,1)，两项基本检查均通过。

试分配后 Available′=(1,0,1)，Allocation0′=(1,1,1)，Need0′=(0,1,0)，其他进程不变。此时 P0、P2 都缺 B，P1 需要 2 个 A 而只有 1 个，P3 需要 2 个 C 而只有 1 个；没有任何进程满足 Need⩽Work，无法找到安全序列。故回滚试分配，使 P0 暂时等待，保留原安全状态。

（4）不等价（1 分）

不安全只表示不能保证满足所有已声明的最大需求；进程未必立即请求全部剩余需求，实际死锁不一定已经发生。

评分标准：（1）公式 1 分，全部向量正确 2 分（部分正确酌情给分）；（2）完整有效序列 1 分，正确 Work 过程 2 分；（3）需求与可用量检查 1 分，试分配结果及无法继续的依据 1 分，结论与回滚 1 分；（4）区分不安全与已死锁 1 分。

课件出处：Sync02 死锁.pdf，PDF 第 9、15—20、34 页。

30. 屏障、条件变量与 IPC 设计（10 分）

（1）一次性屏障（6 分）

```
int count = 0;
mutex m;
condition cv;

barrier_wait() {
    lock(m);
    count++;
    if (count == 4) {
        broadcast(cv);
    } else {
        while (count < 4) {
            wait(cv, m);
        }
    }
    unlock(m);
}
```

m 使计数更新和条件检查互斥，防止丢失更新。count<4 的线程等待时原子地加入等待队列并释放 m，允许其余线程继续到达；最后一个到达者把 count 变为 4 并广播。被唤醒者重新取得 m 后才返回。

Mesa 语义下，唤醒仅意味着等待者可以重新竞争锁，通常应使用 while 重新检查谓词，而非把通知本身当作条件成立的凭据；也可抵御实现允许的虚假唤醒。本题一次性屏障的 count 单调增长，达到 4 后不再减小。若放宽为可复用屏障，不能简单将 count 清零沿用本代码，还需区分不同轮次；本题不要求该扩展。

（2）共享内存与显式同步（4 分）

优先考虑共享内存：把同一物理内存区域映射到两个进程中，双方直接访问数据，减少经内核缓冲区中转的大块数据复制。建立／取消映射等管理操作仍需操作系统参与。

共享内存不会自动提供互斥或生产—消费先后顺序。仍需双方均可使用的信号量，或互斥锁配合条件变量：写者在空位可用时写入，完成后通知读者；读者在数据就绪后读取，并在读完后通知写者复用。协议同时保护元数据和数据可见性，不能只加锁而不判断空／满状态。

评分标准：（1）持锁更新与释放 1 分，未到齐时原子等待且用 while 重查 2 分，最后到达者广播 1 分，解释互斥与重新检查各 1 分；其他正确等价实现可得分。（2）共享内存 1 分，减少中转复制的理由 1 分，说明互斥 1 分，说明数据就绪与缓冲区可复用的条件同步 1 分。

课件出处：Sync03 信号量-条件变量-屏障.pdf，PDF 第 19—20、31—34、40—42 页；Sync04 进程间通信.pdf，PDF 第 24—29、33—35 页。

附：客观题出处与课件索引

页码为 PDF 文件页序，从 1 开始。每题均由课件概念改编；计算数据为本卷自行设定。

题号	出处	题号	出处
1	I1 p.20、50	14	S2 p.15—20
2	I2 p.34—35	15	S4 p.24—29
3	I2 p.36—37	16	P1 p.17—19
4	P1 p.25—28	17	S3 p.6
5	P1 p.9—14	18	S2 p.19—20
6	P2 p.5—7、11—13	19	S3 p.5—10
7	P2 p.20—22	20	S3 p.31—34
8	P3 p.15—16	21	S3 p.19—20
9	P3 p.17—20	22	S3 p.40—42
10	P3 p.22；S2 p.33	23	SX p.5—8
11	S1 p.35—42	24	S4 p.15—18
12	S1 p.29—30	25	S4 p.28
13	SX p.3—4；S3 p.15—17		

I1　Intro01 课程简介.pdf
I2　Intro02 操作系统结构与组成.pdf
P1　Proc01 进程.pdf
P2　Proc02 线程 (1).pdf
P3　Proc03 CPU调度.pdf
S1　Sync01 临界区与锁.pdf
SX　Sync01 临界区与锁-扩展.pdf
S2　Sync02 死锁.pdf
S3　Sync03 信号量-条件变量-屏障.pdf
S4　Sync04 进程间通信.pdf